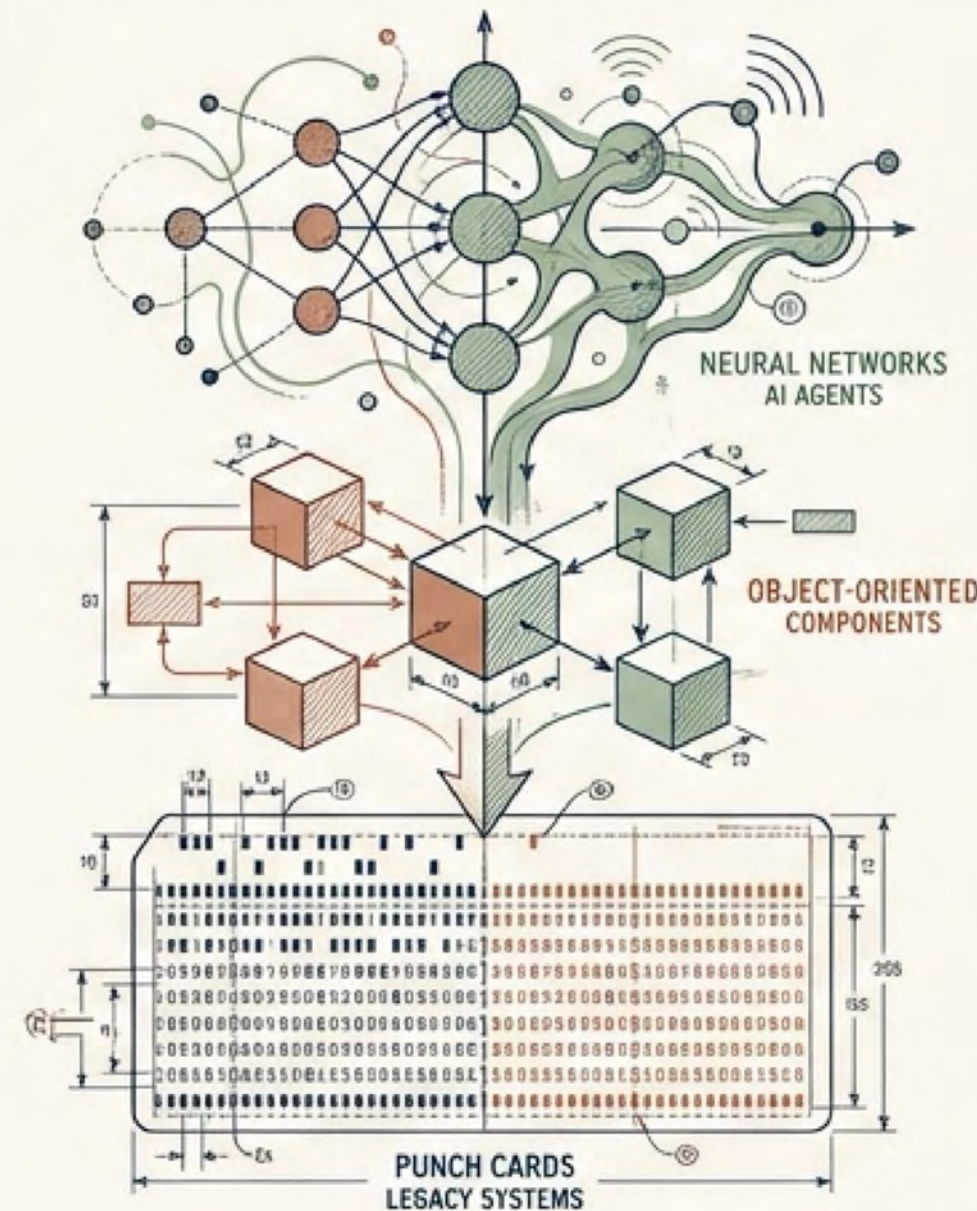


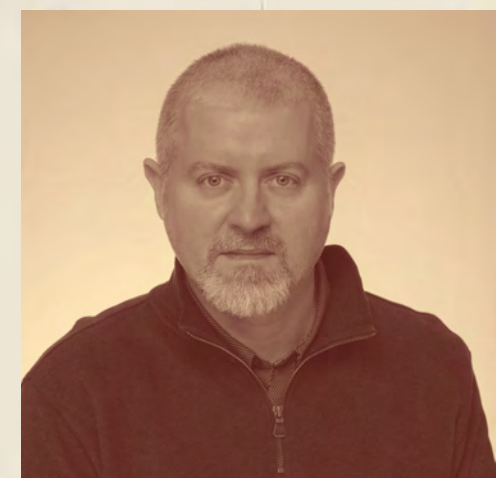
The Third Golden Age of Software Engineering

Pattern recognition for the AI era.



YOUR GUIDE THROUGH THE AGES

A practitioner's perspective on engineering evolution.



OVIDIU EFTIMIE

Senior Director of Engineering, Adobe
Primetime Emmy Engineering Award Recipient

With over **20 years** driving software solutions at scale, Ovidiu leads engineering teams that build and operate the cloud services behind **Adobe Experience Manager**, **GenStudio**, and **Adobe Commerce**. His portfolio spans high-availability systems serving millions: from video analytics and real-time streaming to authentication infrastructure and concurrency monitoring.

BACKGROUND	EXPERTISE	WHY THIS TALK
Master's in Computer Science, Politehnica University of Bucharest . Started as a software engineer in Paris, building systems for French government institutions and enterprises. Joined Adobe in 2007; grew from individual contributor to engineering leadership over 18 years. Built and shipped 1.0 products from the ground up.	Cloud-native architectures at enterprise scale. AWS, Kafka, Spark Streaming , high-availability SLAs, and the full product lifecycle: build, ship, operate. Currently exploring the intersection of GenAI and engineering practice : how AI reshapes the craft of building software, and what stays the same.	Every generation of engineers has faced the same fear: "the new abstraction will replace us." Having lived through multiple such transitions firsthand, this deck traces the pattern across three golden ages and asks: what does history actually teach us about the future of our profession?

CONTEXT // PRESENTER BIO	This talk was delivered at the University of Bucharest on March 3, 2026 .
--------------------------	---

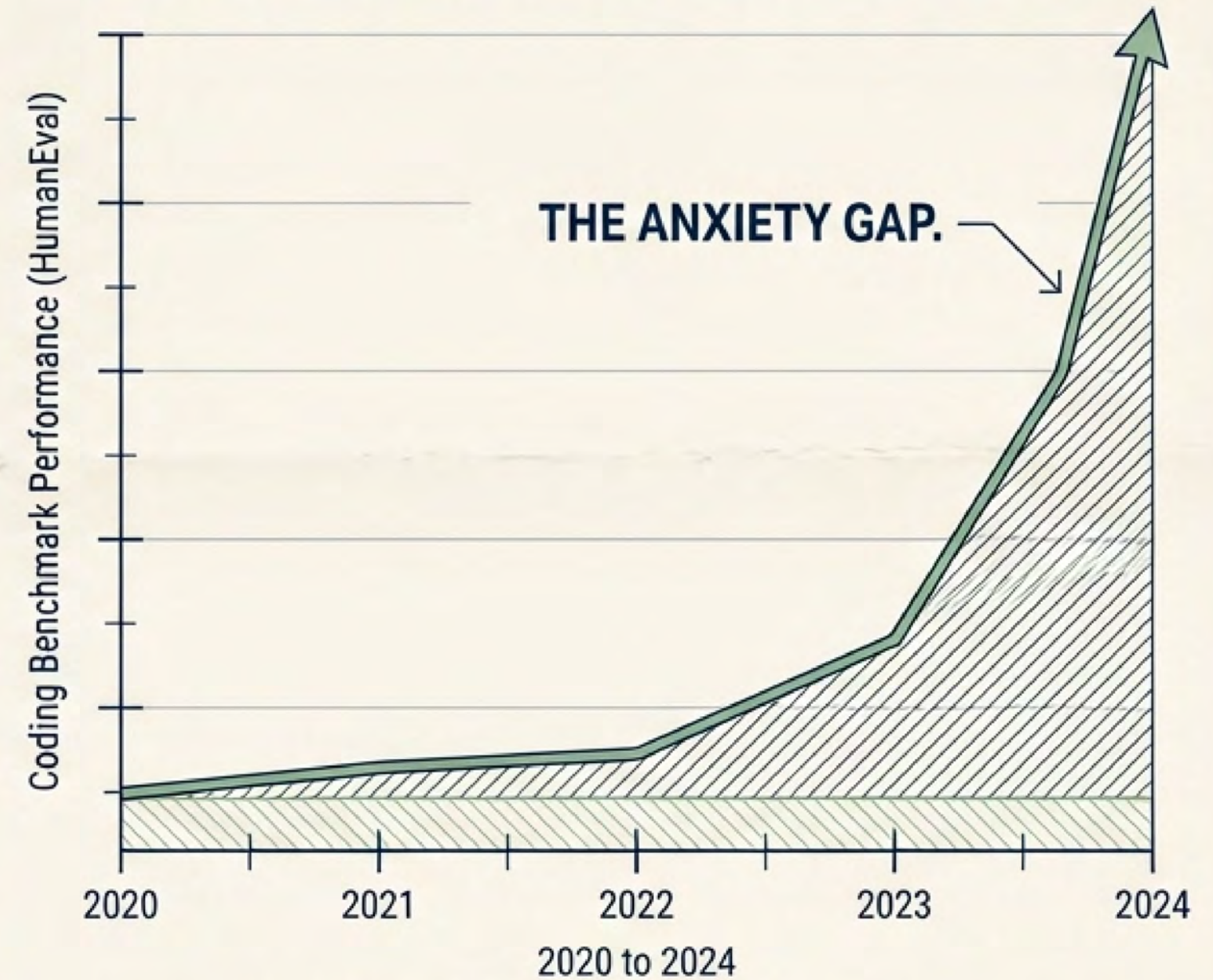
THE EXISTENTIAL DREAD IS REAL.

In the last year, we went from “AI can write decent functions” to “AI can build entire features.” The capability jump is unprecedented. Engineers are asking: Is this the end of the profession?

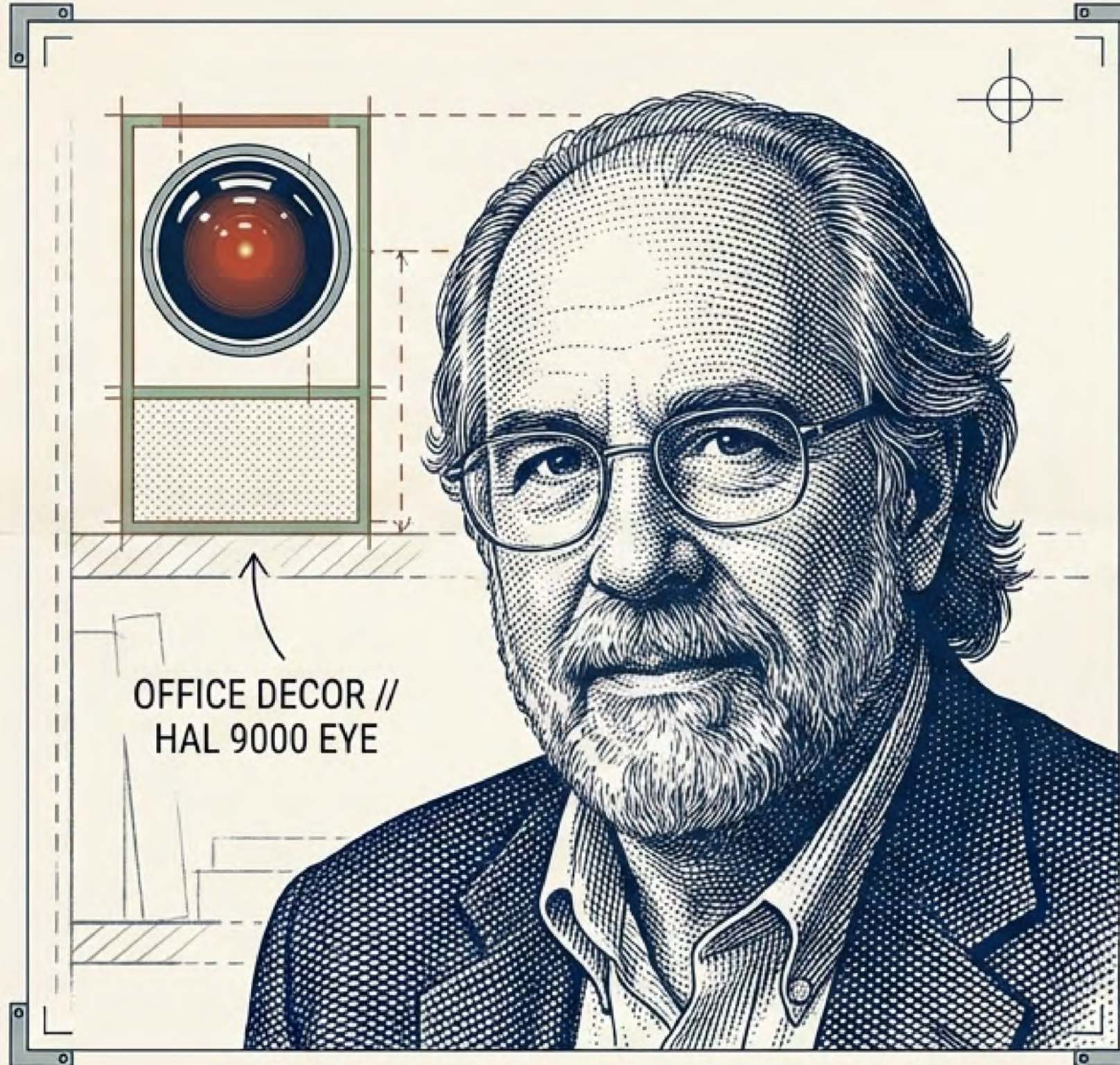
“SOFTWARE ENGINEERING WILL BE AUTOMATABLE IN 12 MONTHS.”

— Dario Amodei, CEO of Anthropic

CODING BENCHMARK PERFORMANCE (HUMAN EVAL)



NOTE: DATA REFLECTS ACCELERATED AI CAPABILITIES AND RAPID ADOPTION.



THE TIME TRAVELER'S PERSPECTIVE.

Why we listen to Grady Booch.

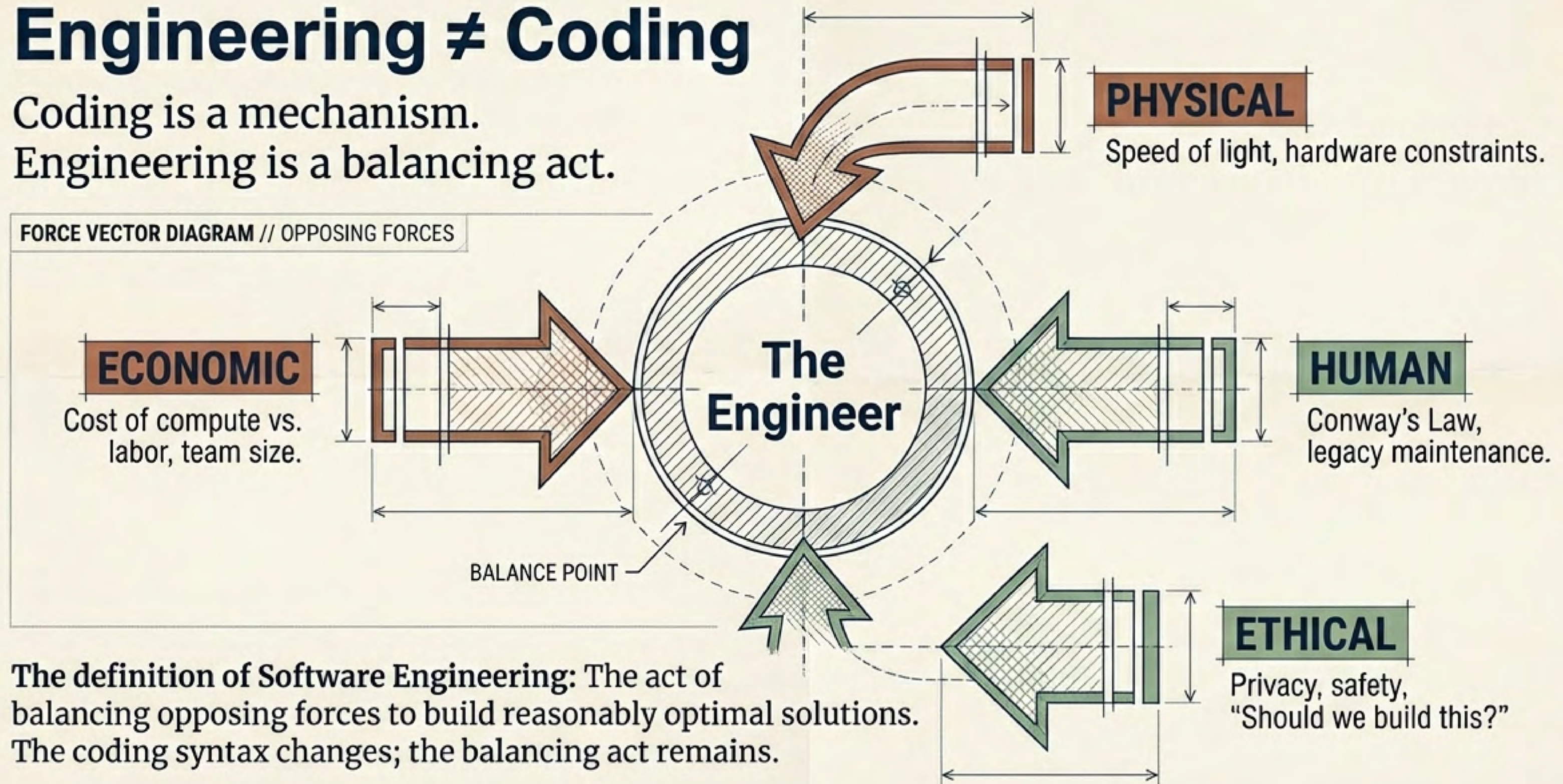
- **ROLE:** IBM Fellow
- **INNOVATION:** Co-creator of the Unified Modeling Language (UML)
- **LEGACY:** Pioneer of Object-Oriented Design (OOD)
- **STATUS:** Witness to every major industry shift since the 1970s

Grady disagrees with the “End of Engineering” narrative. He argues we are entering a Third Golden Age, driven by the same historical force that caused previous disruptions: Rising Abstraction.

Engineering ≠ Coding

Coding is a mechanism.
Engineering is a balancing act.

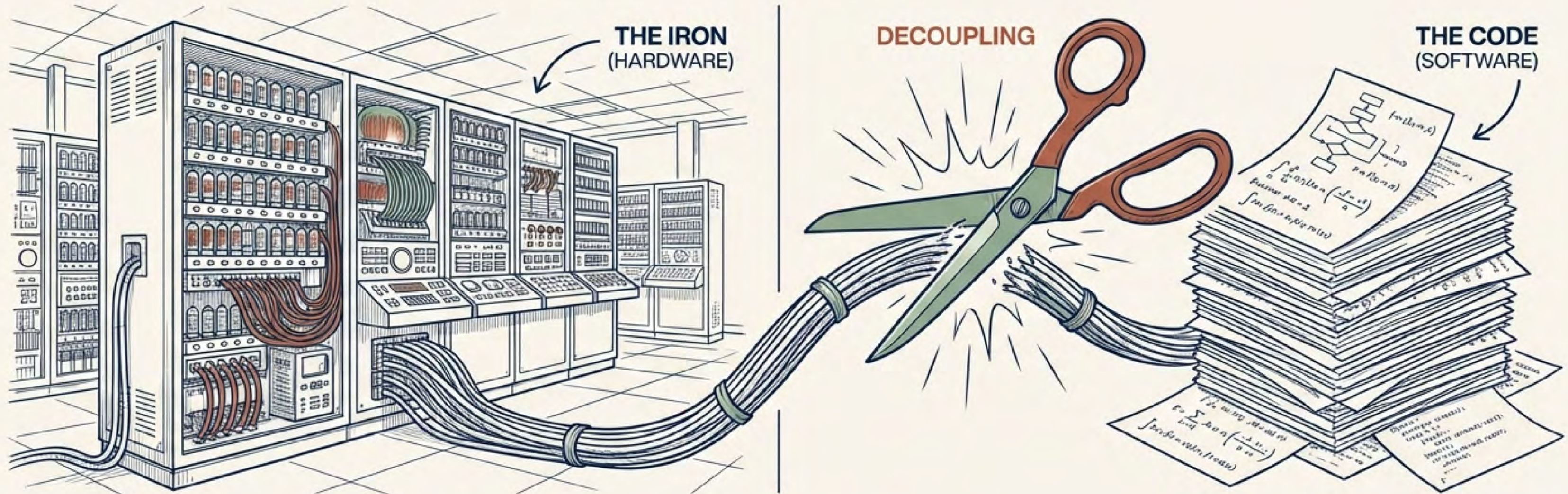
FORCE VECTOR DIAGRAM // OPPOSING FORCES



The definition of Software Engineering: The act of balancing opposing forces to build reasonably optimal solutions.
The coding syntax changes; the balancing act remains.

THE FIRST GOLDEN AGE

Decoupling Code from Iron (1940s – 1970s)



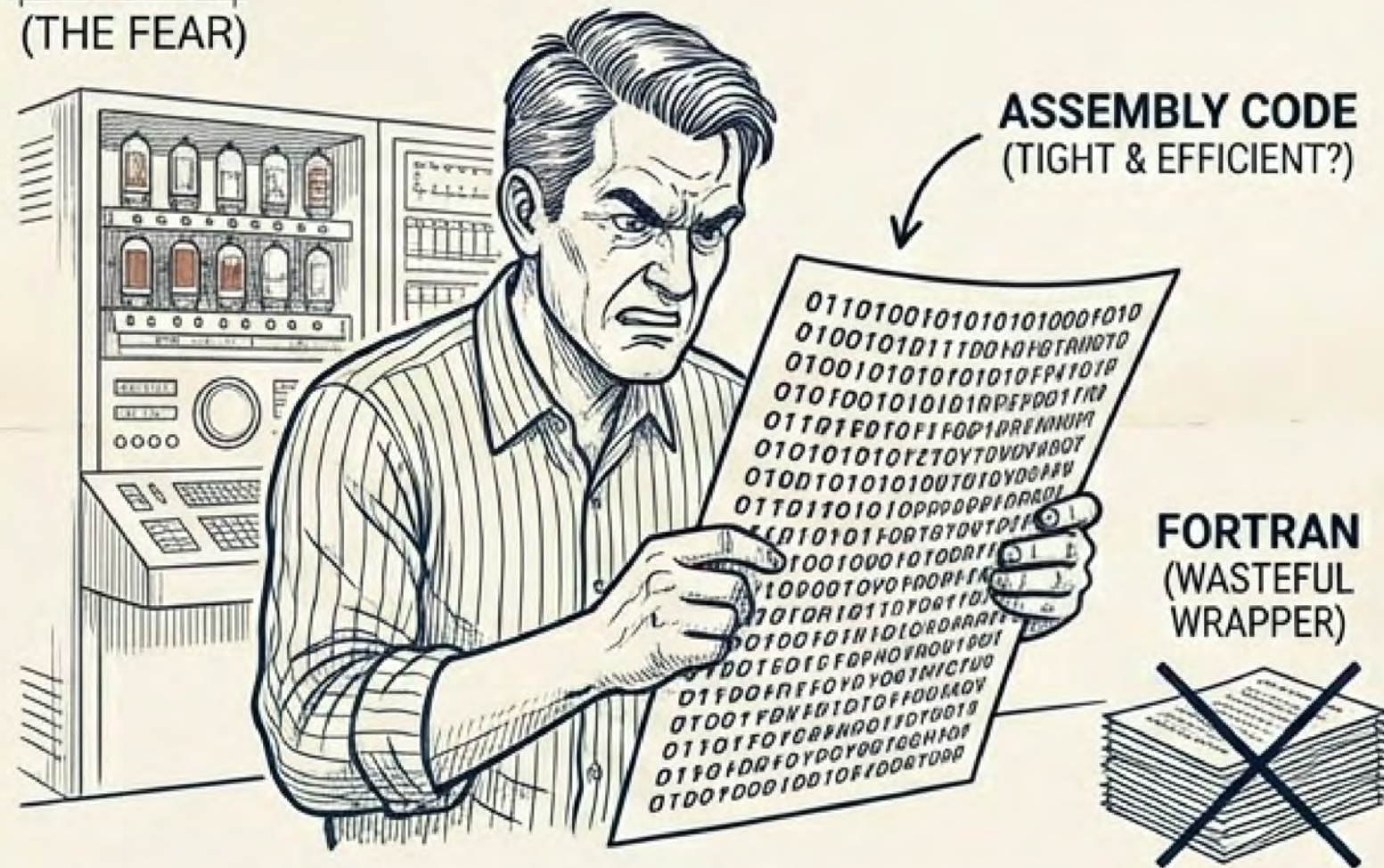
THE TRIGGER: Machines were rare and expensive; humans were cheap. We needed to reuse software across different machines.

THE ABSTRACTION: **Algorithmic Abstraction.** Translating math and formulas (**Verbs**) into machine behavior.

KEY FIGURES: **Margaret Hamilton** (coined "Software Engineering" for Apollo) and **Grace Hopper** (realized software is distinct from hardware).

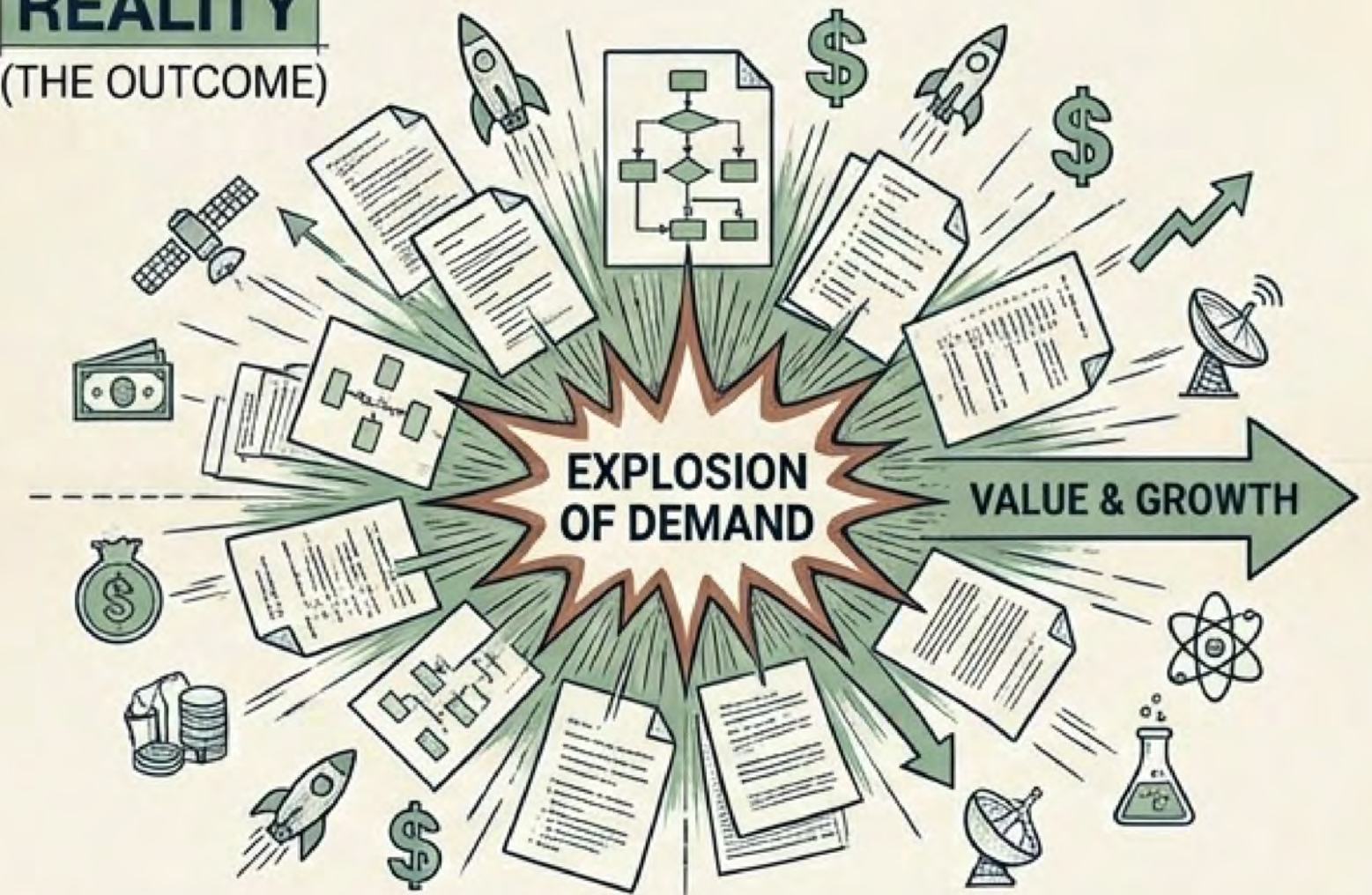
THE FIRST PANIC: "COMPILED WILL KILL EFFICIENCY"

THEN
(THE FEAR)



THE 1950s ARGUMENT: "A machine can never write code as tight or efficient as a human expert." High-Level Languages like Fortran were viewed as wasteful wrappers.

REALITY
(THE OUTCOME)



THE OUTCOME: They were right about efficiency (initially) but wrong about value. Moving up the stack didn't kill jobs; it exploded the demand for software because it became cheaper to produce.

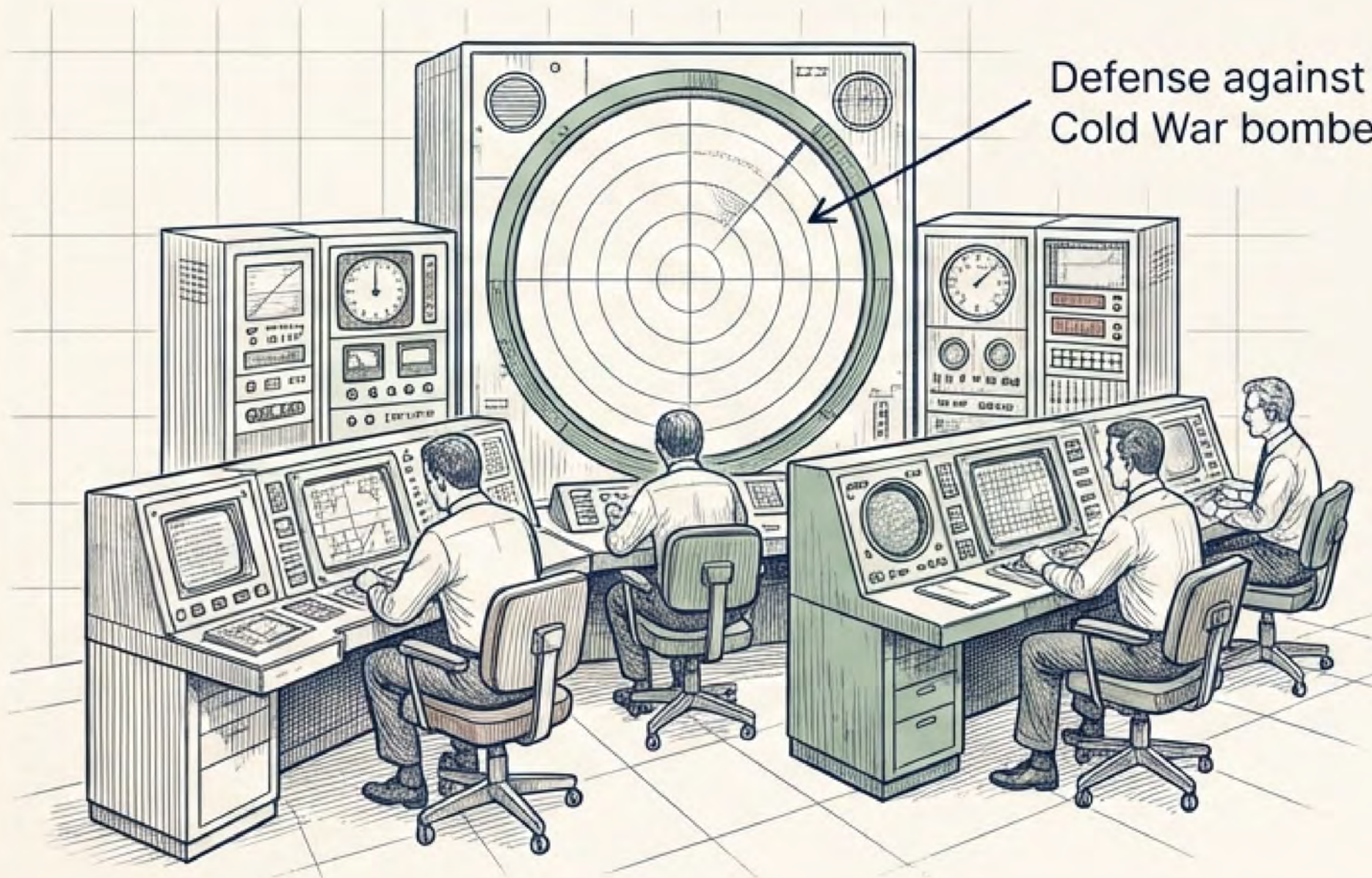
WOVEN UPON THE LOOM OF SORROW

The drivers of the First Age: Commerce and Warfare.

Defense against
Cold War bombers.

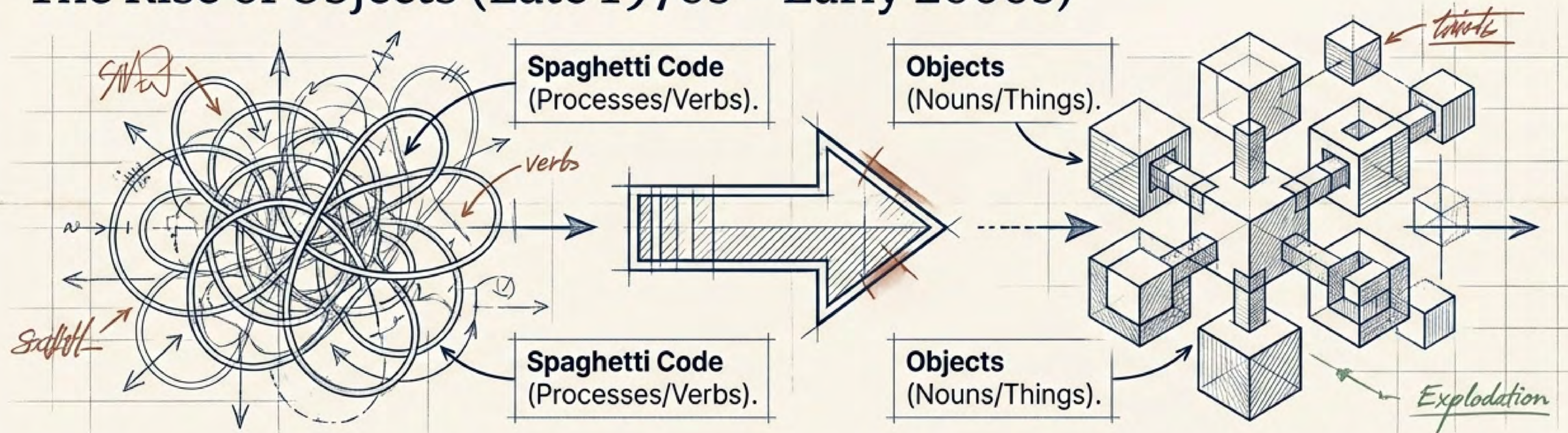
Case Study: SAGE consumed 20–30% of every software developer in the US. It pioneered real-time processing and distributed systems not for profit, but for survival.

Takeaway: Innovation often happens at the “fringe” (defense, space) before hitting the mainstream.



THE SECOND GOLDEN AGE

The Rise of Objects (Late 1970s – Early 2000s)



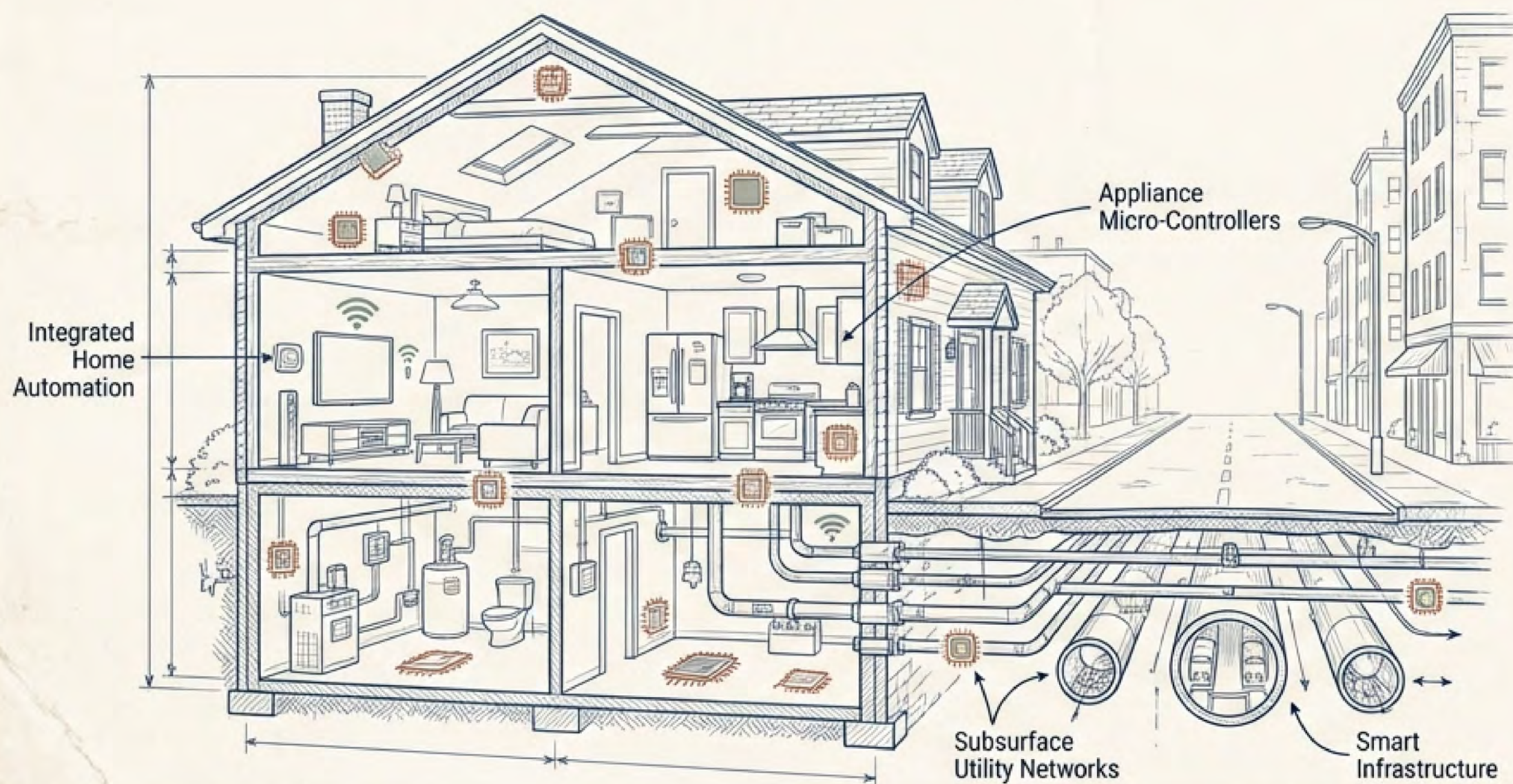
The Trigger: The 'Software Crisis.' Code became too complex to maintain using only algorithmic methods.

The Abstraction: Object-Oriented Design.

The Shift: We stopped organizing code by what it *does* and started organizing by what it *is*. This allowed us to build systems of massive complexity, like GUIs and operating systems.

FILLING THE INTERSTITIAL SPACES OF CIVILIZATION

The drivers of the Second Age: Commerce, Connection, and Ubiquity.



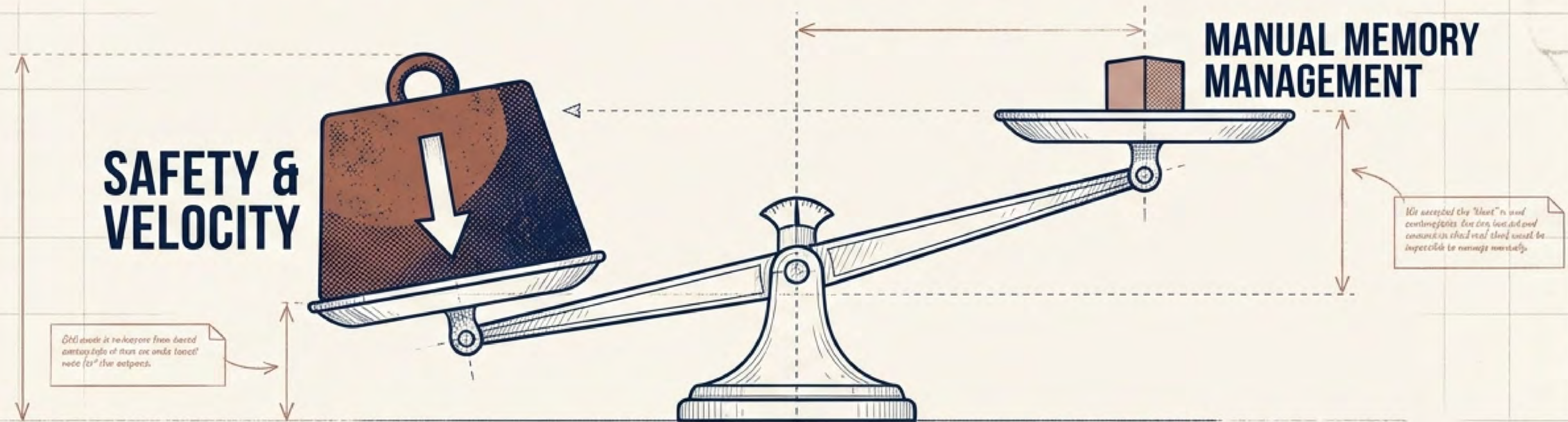
Drivers of the Second Age:

1. **Micro-miniaturization:** The PC revolution allowed hobbyists to “play” for the first time.
2. **The Internet:** Software moved from back-office mainframes to the fabric of daily life.
3. **Open Source:** The culture of sharing industrialized via the web.

Outcome: Software became the invisible “air we breathe,” running everything from banking to coffee makers.

THE SECOND PANIC: "WE ARE LOSING CONTROL"

The shift away from C/Assembly to Managed Runtimes and Java.



The Fear: Moving from C/Assembly to Java and Managed Runtimes.

The Complaint: "Real engineers manage their own memory. These libraries are bloated. We don't know what's happening under the hood."

The Reality: We accepted the "bloat" in exchange for the ability to build massive systems (GUIs, Networks) that would be impossible to manage manually.

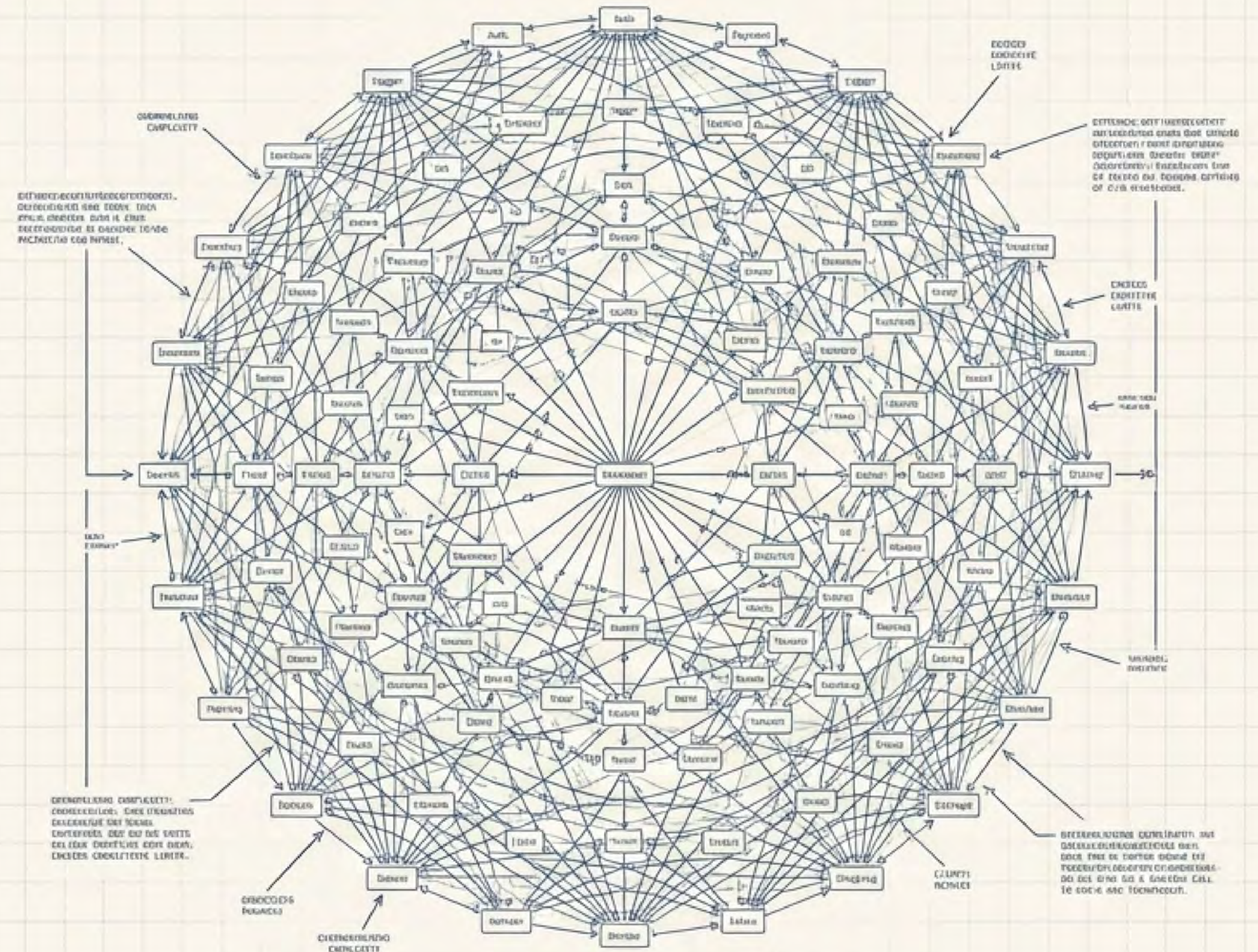
THE THIRD GOLDEN AGE

The Age of Systems (Now)

The Trigger: We are drowning in glue code. The complexity of modern stacks (Kubernetes, Supply Chains) exceeds individual cognitive capacity.

The New Constraint: It is no longer “How do I write this function?” but “How do I secure this supply chain?”

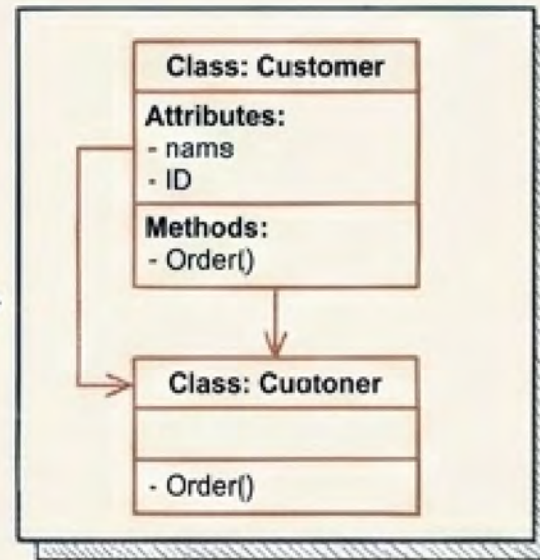
The Abstraction: From Objects to Agents & Systems.



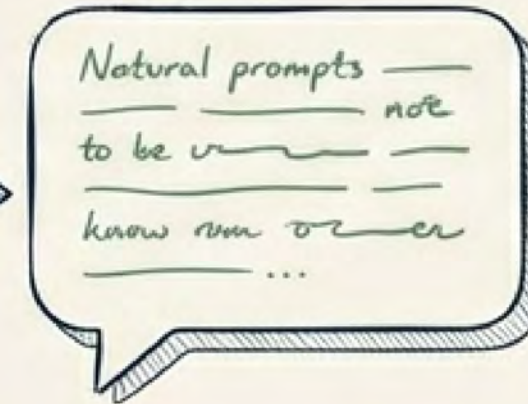
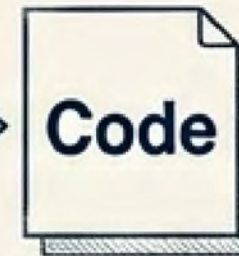
AI is the Interface. English is the Syntax.

$$\sum_{n=0}^n \int_0^{\infty} \frac{f(n-\beta) dx}{x^2}$$

(1950s)



(1990s)



(Now)

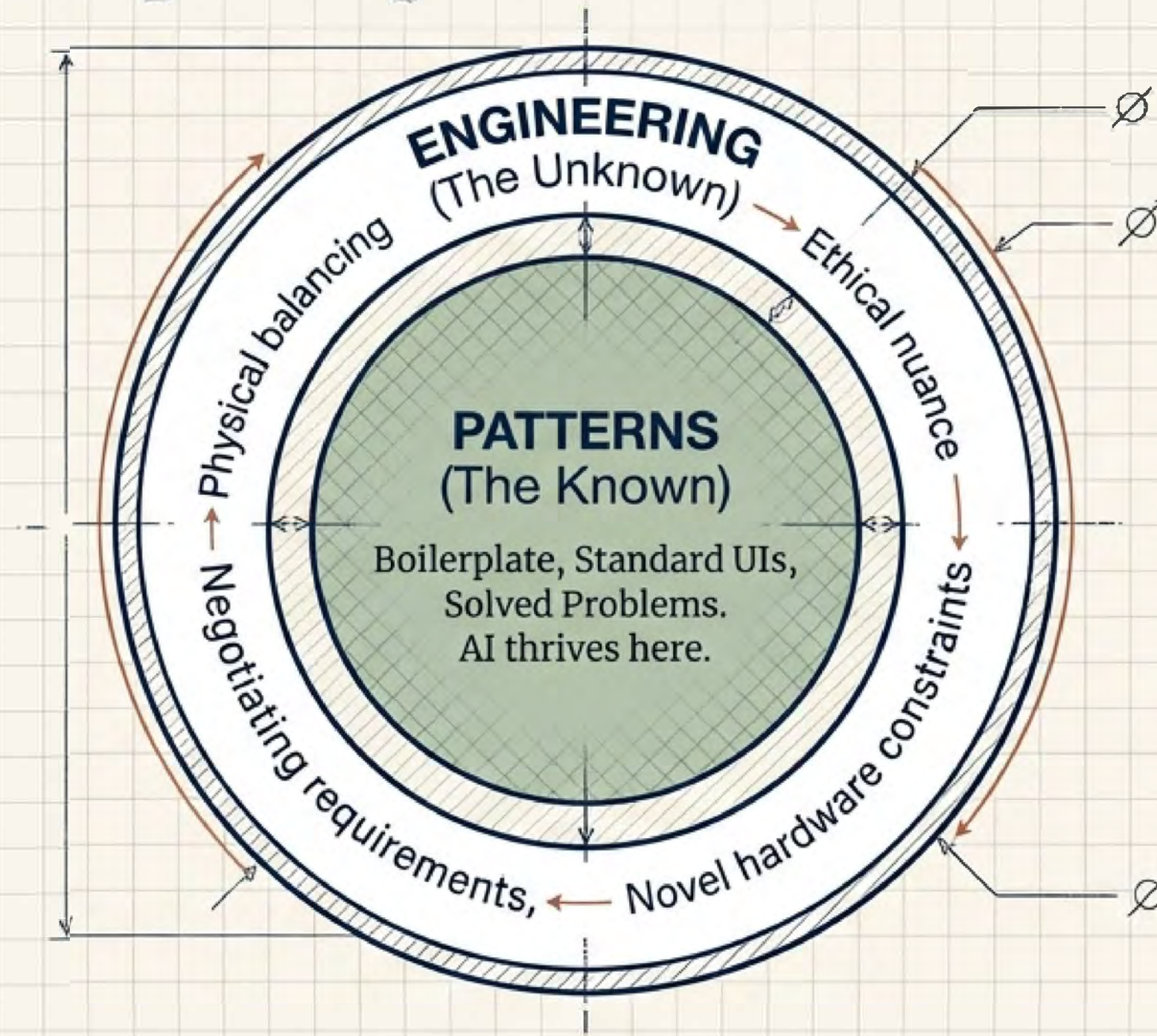


AI bridges the gap between **Human Intent** (ambiguous, nuanced) and **Machine Execution** (precise, binary).

Example: Grady Booch using Cursor to generate D3.js visualizations. He didn't need to learn the library syntax; he needed to understand the *system goal*.

The Rebuttal: “Utter Bullshit.”

Why AI won't automate engineering in 12 months.



— “Fear not, O developers. Your tools are changing, but your problems are not.” — Grady Booch —



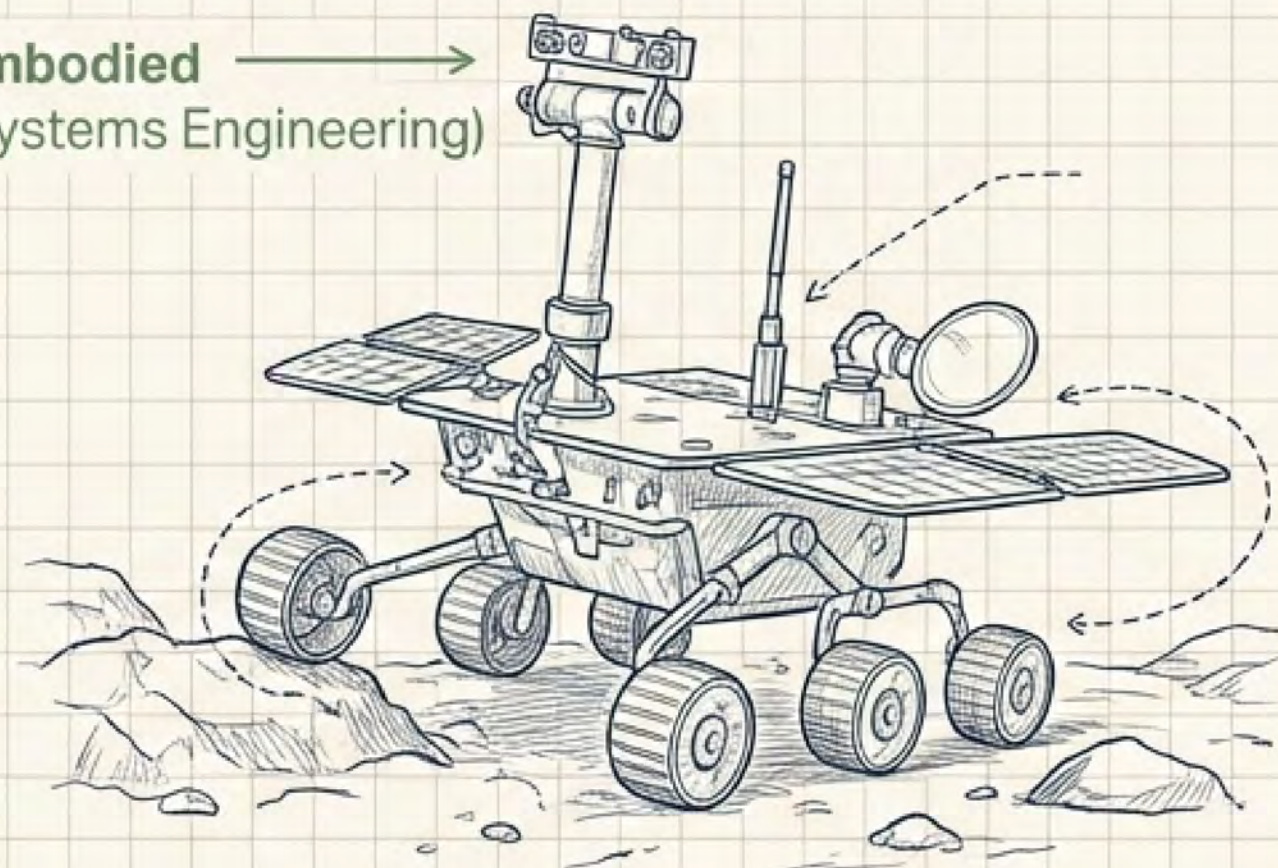
The Limit: Disembodied Intelligence

Disembodied
(LLMs)



A hallucinating AI in a chat window is annoying.

Embodied
(Systems Engineering)



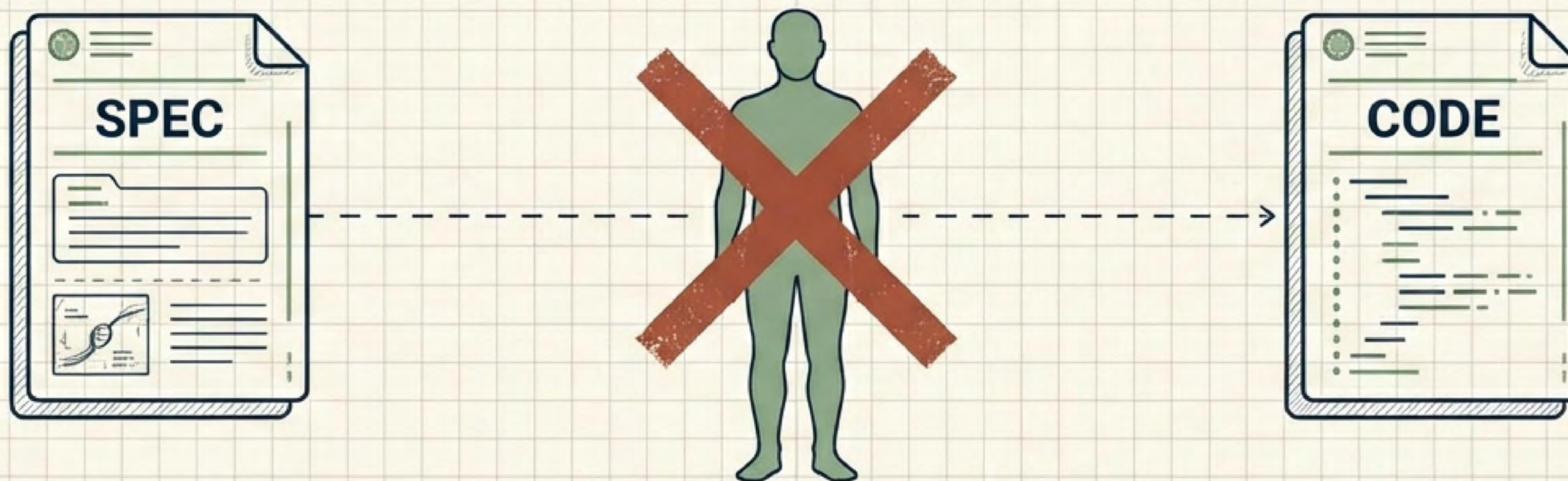
A hallucinating AI in a pacemaker or a Mars rover is catastrophic.

The Reality Check: A hallucinating AI in a chat window is annoying. A hallucinating AI in a pacemaker or a Mars rover is catastrophic.

The Hallway Problem: An AI cannot walk down the hall to ask the hardware engineer why the voltage dropped. Systems Engineering requires 'embodied' understanding of the physical world.



What Dies: The Middleman Work



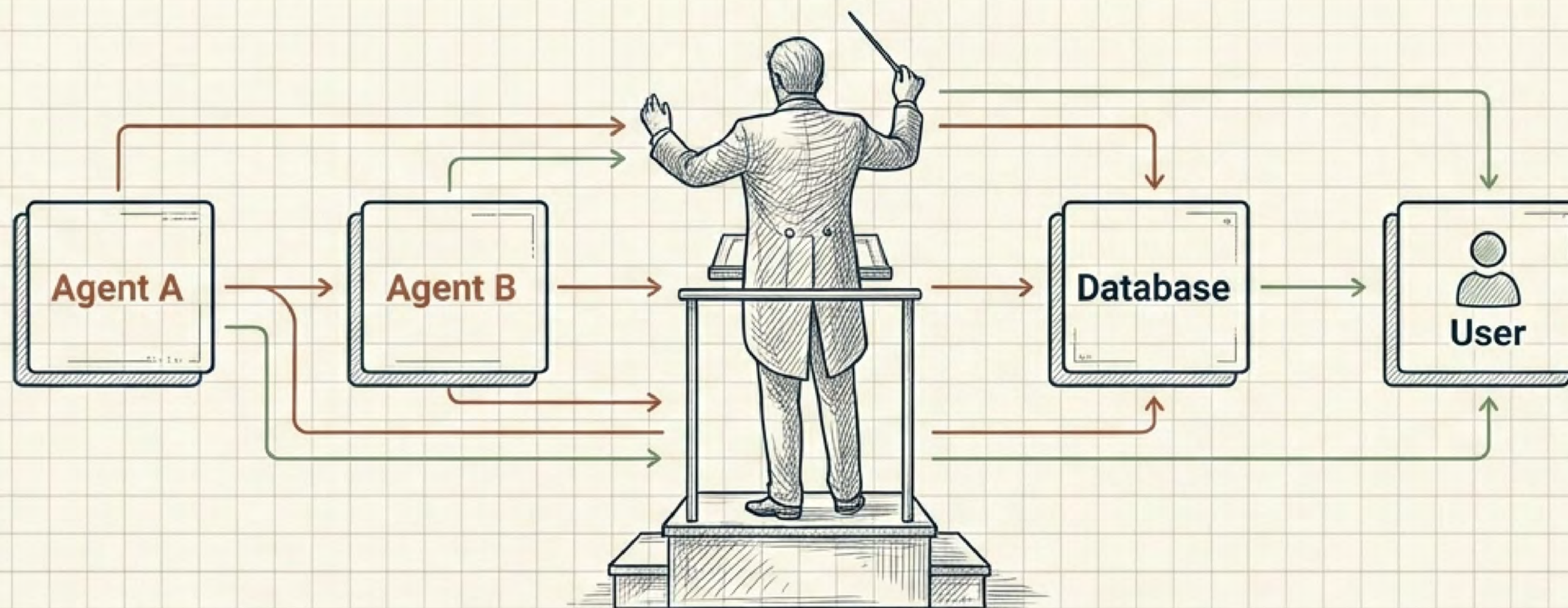
Obsolete Skills:

- Boilerplate generation.
- Translating simple intent to syntax (e.g., 'Write a regex for email').
- Manual pipeline configuration.

Insight: If your job is strictly translating a well-defined spec into code, you are in danger. This is the 'tedium' being automated away.



What Thrives: Systems Thinking

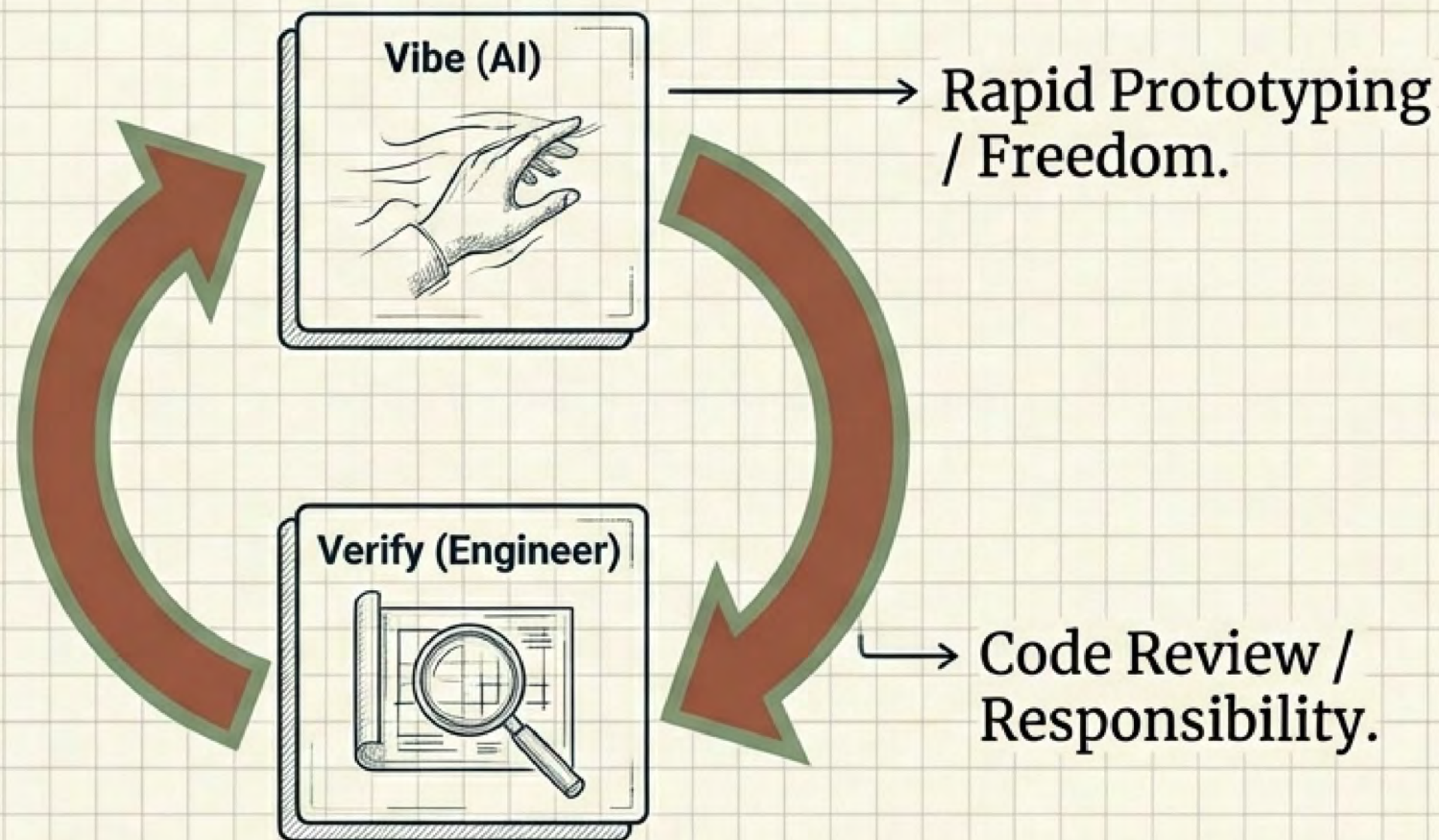


The New Skill Stack:

1. **Orchestration:** Managing societies of agents (Minsky's *Society of Mind*).
2. **Architecture:** Designing for resilience when components are black boxes.
3. **Ethics & Security:** Protecting against supply chain injection and bias.



The New Workflow: Vibe, Then Verify



The Bottleneck: We are generating code faster than we can review it. The ability to *read* and *debug* systems you didn't write is now more valuable than writing from scratch.

Guardrails: Tools like Sonar become essential to check the 'vibe' against reality.



The Hobbyist Renaissance



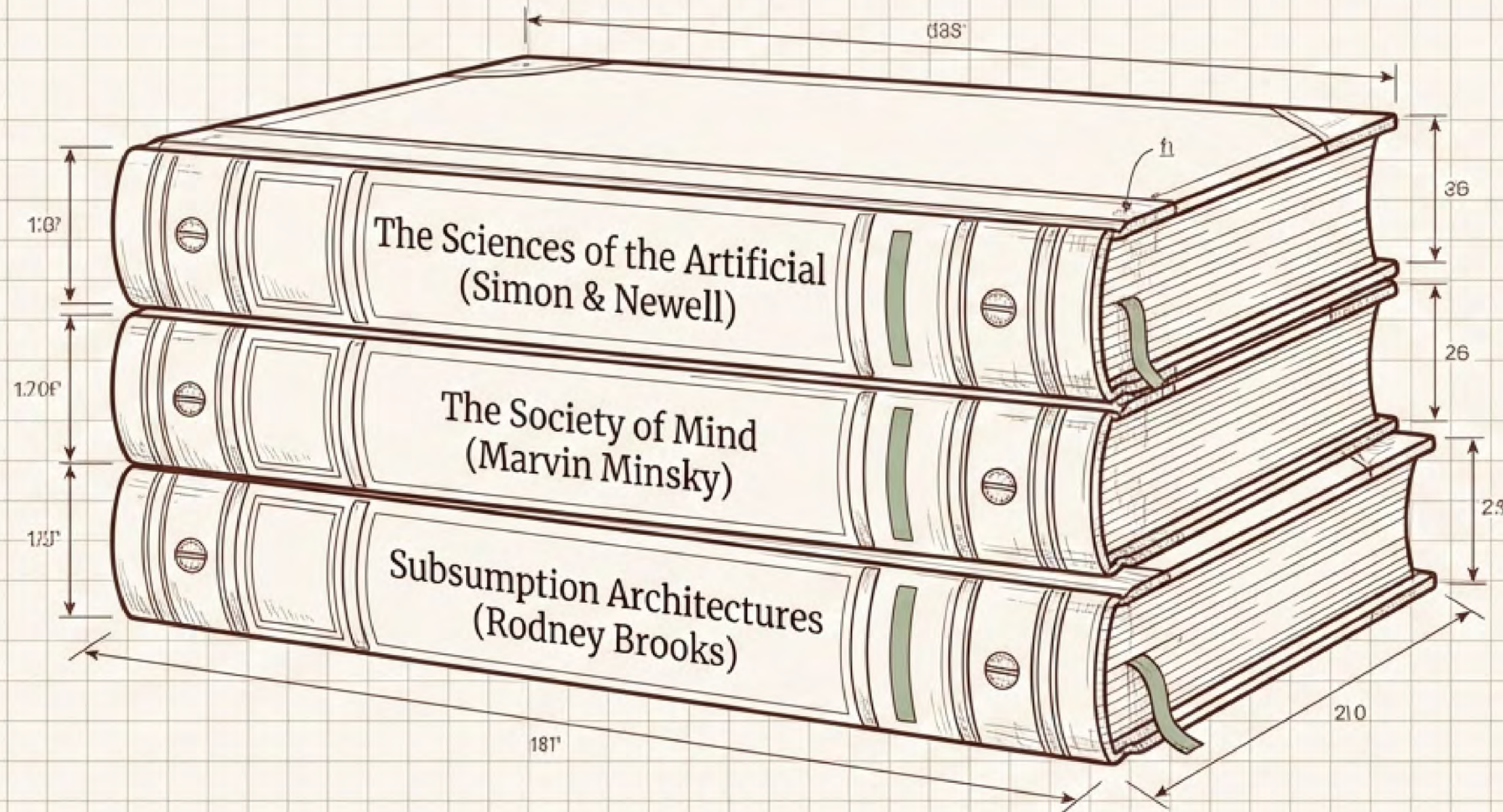
Just like the PC allowed non-engineers to compute, AI allows non-coders to build.

The Expansion: This doesn't shrink the industry; it expands it. We are entering an era of "throwaway software"—custom tools built for an audience of one, previously economically impossible.



Return to the Fundamentals

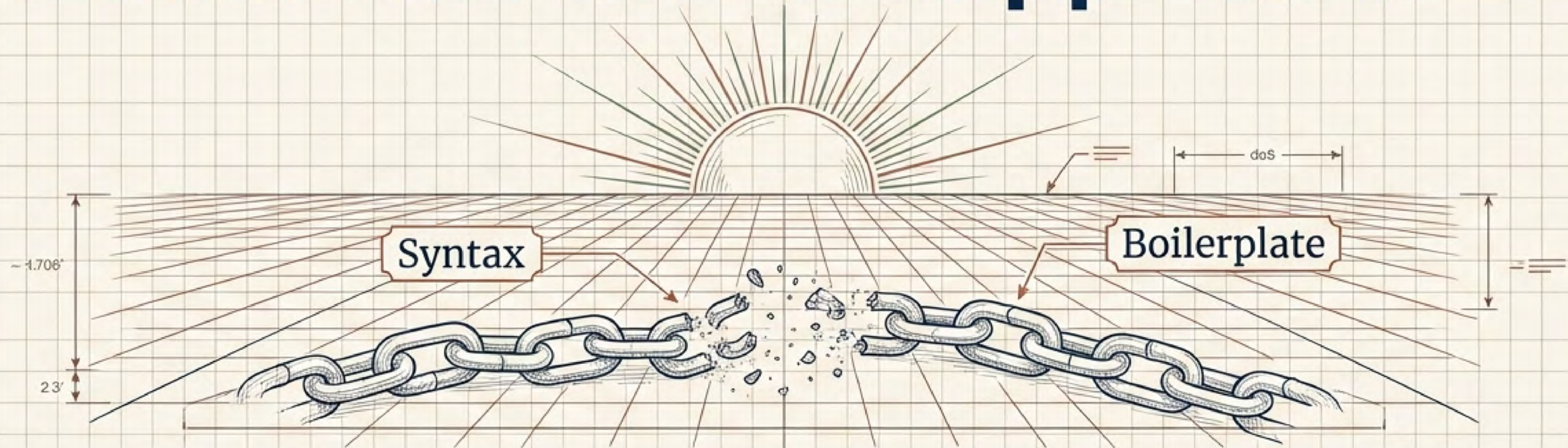
When syntax changes daily, theory remains constant.



Why? These texts describe how complex systems emerge from simple components—exactly what we are building with AI agents today.



Don't Look at the Abyss. Look at the Friction That Just Disappeared.



We are limited only by our imagination and the laws of physics.
The tedium is gone. The Engineering remains.

LEAP.